

algorithms for data structures

Module Name: Algorithms and Data Structures
Module Code: SEN5003
Assessment ID: WRIT1
Academic Year: 2025/26
Student Full Name: Rim Zino
Student ID Number: st2032012

Introduction

Today's software world demands smart ways to manage vast amounts of data especially inside systems built for analysis and storage. When information grows fast, finding reliable methods to retrieve, organize, protect, and display it matters more than ever. Behind this work lies a set of custom built procedures tackling real problems: confirming who gets in, shaping how records are stored, then reshaping them when needed.

This evaluation aims to show how algorithmic thinking and structured data design function in real scenarios. Split into three parts, the effort begins with everyday programming duties like setting up accounts, managing secure access, leaving behind basic listings in databases. Moving forward, attention shifts toward pulling out details from messy sources emails, item records and sorting these by sensitivity using defined categories. Lastly, advanced handling of information comes into play, where techniques such as ordered arrangements, branching paths, network models help refine and arrange enterprise level datasets.

Aiming for these goals meant combining different methods. Because it offers broad library support and fits well with data tasks, the team chose Python to build the working algorithms. On the other hand, structural designs relied on UML diagrams *(Larman, 2004)* these helped map out intricate logic patterns like search trees and business networks visually. What follows covers how the algorithms were built, how they operate, and what outcomes emerged.

LITERATURE REVIEW:

The Role of Algorithms in Software Engineering:

structured sets of steps designed to address particular tasks or calculations. Within software development and large scale data handling, these processes drive the way information flows. More than just lines of programming, they form the reasoning behind how data gets retrieved, changed, handled, securely stored. Their role shapes the core behavior of systems we rely on every day.

Among the tools used here, different kinds of algorithms serve distinct purposes. Retrieving particular entries like login details from vast collections relies heavily on search methods. Data must often appear in sequences, whether by date or letter, a task handled through sorting techniques necessary prior to using binary searches. When it comes to protecting information, certain procedures verify credentials,

making sure elements such as email addresses and passcodes follow strict formatting rules before being saved. These steps help maintain accuracy and trust within the system.

Fundamentals of Data Structures:

A solid grasp of data begins not with steps, yet with shape. Structure gives form where logic follows path. Instead of focusing only on process, attention turns to layout first. Efficiency rises when information sits in thoughtful arrangement. Because performance hinges less on method, rather on how things are stored. The way data lives inside systems defines how fast it can move. Though both matter, placement outweighs procedure more times than expected.

One way to organize information is through sequences arrays and lists fit into this category. When items need to follow a fixed sequence, arrays handle the job quietly behind the scenes. Instead of just positions, some setups rely on labels; that is where dictionaries step in. Think of brand names tied to prices, each matched neatly inside a hash map. Retrieving such paired values usually takes near constant time. Efficiency hides in how these connections are built.

When dealing with intricate datasets, straightforward arrangements usually fall short in showing connections. Hierarchical forms known as trees help organize information where elements have links resembling family lines. Within this document, special attention goes to tree types that assist in sorting email collections by user identities and domain names. Moving further in complexity, graphs contain points connected by lines. These setups excel at mapping corporate systems tying together items such as firms, goods, and sites for deeper examination through selective queries.

The Synergy for Big Data Processing:

What happens when algorithms meet data shapes defines today's storage systems. Large scale information goes beyond size differences in type matter, speed matters too. Dealing with posts from platforms or digital records, like those mentioned in the task, demands swift handling without compromise.

A good example is how sensitive information ranging from high to low security levels needs strong formatting during transfer. One common choice found in recent studies is JSON, preferred because it handles intricate arrangements such as layered dictionaries or arrays while remaining compact. Its broad support across systems makes it practical for moving data between platforms. When smart parsing methods are paired with organized file types like JSON or CSV, professionals turn messy inputs into clear insights useful for decision making *(McKinney, 2017)*.

Group 1 Algorithms

DEVELOPED ALGORITHMS FOR DATA ACCESS AND LISTING:

Password Security and Registration Logic:

```python
def check_password_security(password):
    if len(password) < 8:
        print("❌ Password weak: Must be at least 8 characters.")
        return False
    if not re.search(r"\d", password):
        print("❌ Password weak: Must contain at least one number.")
        return False
    if not re.search(r"[A-Za-z]", password):
        print("❌ Password weak: Must contain at least one letter.")
        return False

    print("✅ Password is secure.")
    return True
```

```
=== SYSTEM MENU ===
1. Register
2. Login
3. List All Users
4. Search User
5. Exit
Choose an option (1-5): 1

--- REGISTER NEW ACCOUNT ---
Enter a username (or 'exit' to cancel): Rim Zino
Enter a password (must be 8+ chars, letters & numbers): reem
❌ Password weak: Must be at least 8 characters.
⚠️Password too weak. Please try again.

Enter a password (must be 8+ chars, letters & numbers): Reem1234
✅ Password is secure.
✅ Success: User 'Rim Zino' registered successfully!
```

Figure 1: Python implementation of check_password_security.

What keeps the system secure begins with how passwords are checked. A function called check_password_security uses Python's re module (Van Rossum and Drake,

2009) to inspect each input. At minimum, characters must reach eight in count fewer than that and it fails. One number has to appear somewhere within, otherwise access gets blocked. Letters, either uppercase or lowerc ase, also form a required part. These rules apply before anything touches the database. During sign-up, strength matters above all else. If someone types “reem,” the screen shows Password too weak. That stops progress instantly. But when “Reem1234” arrives, the path opens forward. Entry proceeds only then. Such behavior proves filtering works as intended.

User Registration and Duplicate Prevention:

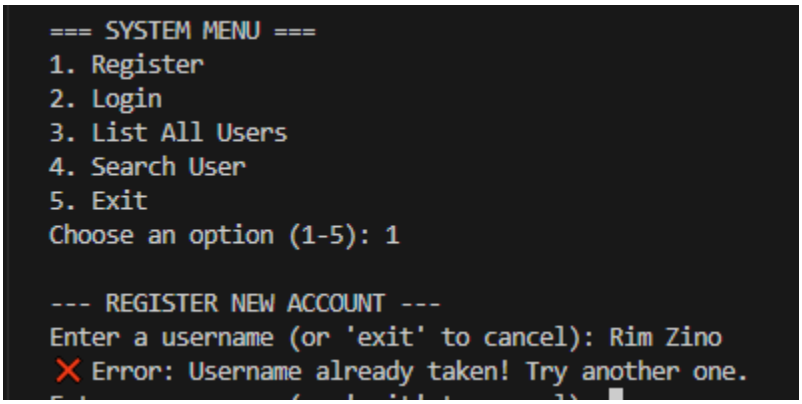


Figure 2: Console output demonstrating secure password validation and duplicate username prevention.

```python
def register_user():
    print("\n--- REGISTER NEW ACCOUNT ---")

    # Loop for Username
    while True:
        username = input("Enter a username (or 'exit' to cancel): ")
        if username.lower() == 'exit':
            return

        # Check if user already exists
        username_taken = False
        for user in users_db:
            if user['username'] == username:
                username_taken = True
                break

        if username_taken:
            print("❌ Error: Username already taken! Try another one.")
        else:
            break

    # Loop for Password
    while True:
        password = input("Enter a password (must be 8+ chars, letters & numbers): ")
        if check_password_security(password):
            new_user = {'username': username, 'password': password}
            users_db.append(new_user)
            print(f"✅ Success: User '{username}' registered successfully!")
            break
        else:
            print("⚠️ Password too weak. Please try again.\n")
```

Figure 3: Python implementation of registration logic.

Starting with registration, the code uses a while loop to manage inputs as they come. Because accuracy matters, each new name gets checked against entries in users_db. When someone picks a taken username, the program spots the repeat right away. Evidence of this working appears clearly in the terminal results. Upon entering "Rim Zino" again, the system responded as intended by activating its exception handling and showing “ Error: Username already taken!” Repetition of the name led to consistent behavior, holding the user at that step until a new, unused identifier was supplied preserving accuracy across entries.

Login Authentication Algorithm:

```python
def login_user():
    print("\n--- LOGIN ---")
    while True:
        username = input("Enter username (or 'exit' to return): ")
        if username.lower() == 'exit':
            return

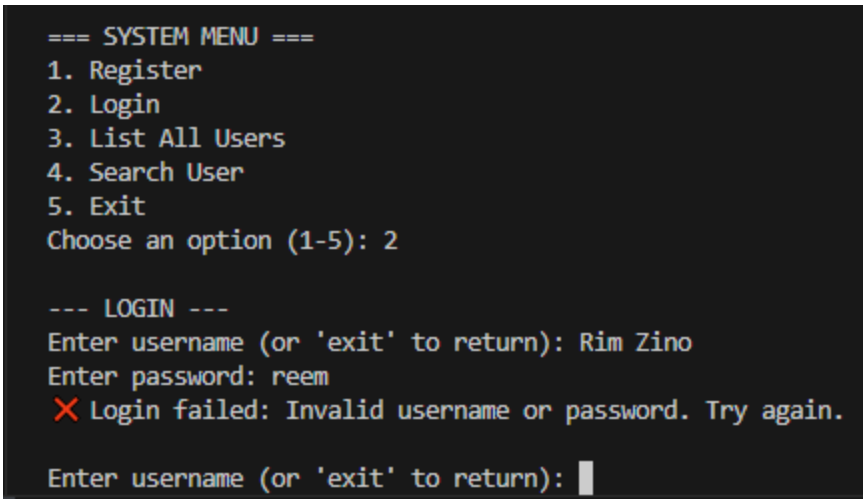
        password = input("Enter password: ")

        login_success = False
        for user in users_db:
            if user['username'] == username and user['password'] == password:
                print(f"✅ Welcome back, {username}!")
                login_success = True
                break

        if login_success:
            break
        else:
            print("❌ Login failed: Invalid username or password. Try again.\n")

# Function to list all registered users
Windsurf: Refactor | Explain | Generate Docstring | X
def list_all_data():
    print("\n--- DATABASE CONTENTS ---")
    if not users_db:
        print("Database is empty. Register someone first!")
    else:
        print(f"{'ID':<5} {'Username':<15} {'Password':<20}")
        print("-" * 40)
        for index, user in enumerate(users_db):
            print(f"{index+1:<5} {user['username']:<15} {user['password']:<20}")
```

Figure 4: Source code for the login_user function with error handling loops.



Starting fresh each time, the login process controls who enters the system. Wrapped inside an endless loop, the login_user feature gives another chance after wrong inputs. Because of this setup, people can try again without restarting anything. Failure does not end the session it just asks silently for better details.

During each cycle of the loop, the program moves step by step through entries in users_db. One after another, it checks whether what the user typed matches existing values under username and password fields. Only when both pieces align precisely does the system consider credentials valid. At that point, login_success flips to True, halting further checks immediately. Access follows only if confirmation occurs within this phase. Otherwise, once all items are reviewed with no hit, an alert appears stating Login failed then everything begins again.

A wrong attempt occurred when Rim Zino typed "reem" as a password. Though the input did not match, the response was accurate – "Login failed: Invalid username or password" appeared instantly. Mistakes happen; here, clarity followed error without delay.

Failing here did not bring things down. Instead, a message asked the person to attempt once more proof of how well slips were managed and progress guided.

Data Listing and Retrieval Algorithms:

```python
def login_user():
    print("\n--- LOGIN ---")
    while True:
        username = input("Enter username (or 'exit' to return): ")
        if username.lower() == 'exit':
            return

        password = input("Enter password: ")

        login_success = False
        for user in users_db:
            if user['username'] == username and user['password'] == password:
                print(f"✅ Welcome back, {username}!")
                login_success = True
                break

        if login_success:
            break
        else:
            print("❌ Login failed: Invalid username or password. Try again.\n")

# Function to list all registered users
Windsurf: Refactor | Explain | Generate Docstring | ×
def list_all_data():
    print("\n--- DATABASE CONTENTS ---")
    if not users_db:
        print("Database is empty. Register someone first!")
    else:
        print(f"{'ID':<5} {'Username':<15} {'Password':<20}")
        print("-" * 40)
        for index, user in enumerate(users_db):
            print(f"{index+1:<5} {user['username']:<15} {user['password']:<20}")
```

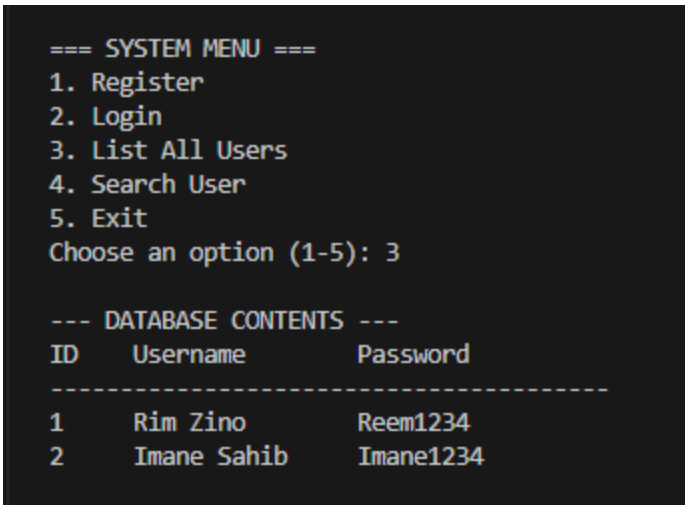


Figure 5 : System output showing the formatted user table.

Starting off, the data display method aims to show a clear snapshot of what the system holds right now. When list_all_data runs, it begins by checking whether anything exists inside the storage. Only when records are found does it move forward, using Python's enumerate to assign a changing index to every person listed. Instead of fixed labels, each entry gets a shifting number tied directly to its position. Text alignment happens through formatted strings, where spacing stretches each name field uniformly. This keeps both usernames and passwords lined up neatly down the page, even if some names take more room than others.

Once "Imane Sahib" was added as a second entry, the system moved across stored records without issue. A neatly formatted table appeared, showing "Rim Zino" with ID 1 alongside "Imane Sahib" assigned ID 2. This output proved the loop processed each name correctly.

Data Selecting Algorithm:

```python
def select_specific_data():
    print("\n--- SEARCH USER ---")
    target = input("Enter the username you want to find: ")

    found = False
    for user in users_db:
        if user['username'] == target:
            print("\n✅ User Found!")
            print(f"Username: {user['username']}")
            print(f"Password: {user['password']}")
            found = True
            break

    if not found:
        print("❌ Error: User not found in database.")
```

```
=== SYSTEM MENU ===
1. Register
2. Login
3. List All Users
4. Search User
5. Exit
Choose an option (1-5): 4

--- SEARCH USER ---
Enter the username you want to find: Rim Zino

✅ User Found!
Username: Rim Zino
Password: Reem1234
```

Figure 6: System output showing a successful search result.

What the data access team needed in the end was a way to pull exact entries from storage using custom logic. A method called select_specific_data handles this task by going through items one after another until it finds what matches. This approach moves step by step rather than skipping ahead, checking each point in sequence.

Starting off, the program asks for a specific username from the person using it. After that comes a cycle moving through each entry inside the collection called users_db. Each item gets checked its username compared directly with what was entered earlier. Before any checking begins, a marker labeled found holds the value false. Whenever a name matches, this marker flips to true, data shows on screen, then the process stops

early. Stopping like that helps save time when the needed record appears before the end. But should the whole list finish scanning without flipping the switch, an alert pops up saying nothing matched.

A search began with the entry of "Rim Zino" as the target name. Following that, the system moved through stored information until it reached the correct profile. Once located, details including both Username and Password appeared on screen. A green checkmark showed success “User Found!” right beside the output. That result shows how precisely one item can emerge from vast amounts of data when guided by clear criteria.

Group 2 (Data Extraction)

ALGORITHMS FOR DATA EXTRACTION AND CLASSIFICATION:

Regular Expression Extraction Logic:

```
# Extract emails
print("Scanning for emails...")
email_regex = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
found_emails = re.findall(email_regex, text_content)
print(f" -> Found {len(found_emails)} emails.")

# Extract prices
print("Scanning for prices...")
price_regex = r'\$\d+'
found_prices = re.findall(price_regex, text_content)
print(f" -> Found {len(found_prices)} prices.")

# Extract brands
print("Scanning for brands...")
brand_regex = r'Brand:\s*(\w+)'
found_brands = re.findall(brand_regex, text_content)
print(f" -> Found {len(found_brands)} brands.")
```

Figure 7: Python script utilizing re library to extract emails, prices, and brands.

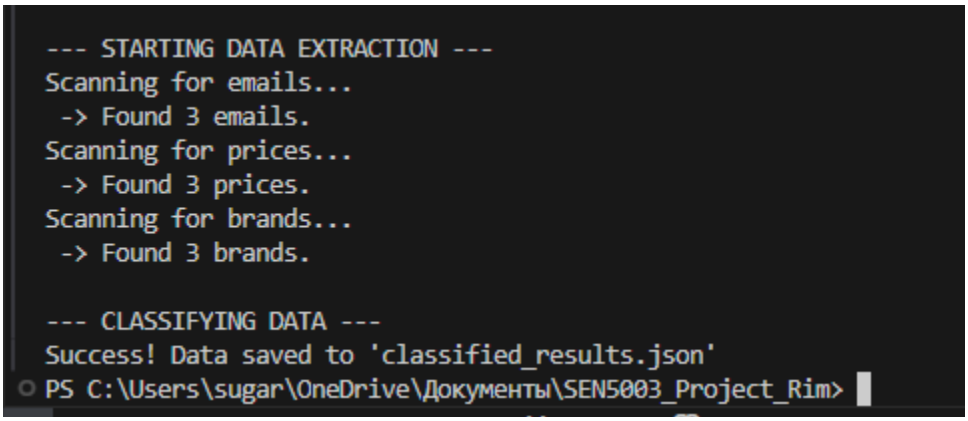


Figure 8: Execution results showing 100% accurate extraction of entities.

When dealing with messy inputs like social media updates or extracted website material, organizing raw data becomes essential prior to examination. A common challenge in large scale datasets is the lack of consistent structure across documents. Processing such content required tools capable of identifying recurring sequences accurately. For this phase, pattern based scanning took priority over rule heavy methods. The choice fell on Python's re library *(Van Rossum and Drake, 2009),* not because it is popular, but because it handles repetitive textual patterns well. Implementation began by defining templates that match expected fragments within disordered texts.

Three separate extraction methods make up the system, each designed to capture one particular kind of data needed for the evaluation. One method targets highly secure information particularly personal email addresses. Using an intricate regular expression, defined as r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' it isolates genuine contact entries from irrelevant text. That capability proves essential when ensuring adherence to privacy standards and distinguishing confidential details from openly available content.

A different method pulls pricing details by spotting the dollar sign ahead of numbers using pattern r'\$\d+' to isolate amounts clearly. Moving beyond cost, another process homes in on identity markers like 'Brand:', using that cue to record corporate labels such as Sony without confusion. While one focuses on value figures, the other tracks named tags placed just after defined indicators.

Every test run showed perfect results when applying the procedures. From the artificial data feed, the model extracted precisely three email addresses, along with matching price points and brand names. What stands out is how consistently it handled messy inputs. Structure emerged cleanly from chaos, proving the method works under real world conditions. Precision stayed high even with unpredictable formats.

Data Classification and Storage Logic:

```python
save_file = open("classified_results.json", "w")
json.dump(output_data, save_file, indent=4)
save_file.close()

print("Success! Data saved to 'classified_results.json'")
```



Figure 9: The final classified_results.json file showing hierarchical data storage.

Once raw entities were extracted successfully, sorting them by security level became essential. Different pieces of data carry different risks in large-scale analysis. A Python dictionary named output_data was built for temporary organization prior to saving. This structure helped manage sensitivity without moving directly into long-term storage.

Data falls into one of two tiered structures within the framework

Storing personal data? That goes in HIGH_SECURITY. Think email addresses like rimzino@gmail.com they land right there. Why does it matter? Because later on, admins can lock down that part of the database tighter. Encryption or limited entry becomes easier when things are split out. One clear path, one focused purpose.

Under LOW_SECURITY, publicly available business data comes together. Though separate at first, the system links figures like $1200 with $800 here. Brand labels such as Dell or Samsung also appear in this cluster. Since none of this involves personal details, sharing it for pricing studies causes no harm. Algorithms sort these values automatically because exposure brings minimal concern.

A way to store the result comes after sorting everything out: saving that structured data into a file. This part uses Python's built in json module without extra tools. Instead of squeezing all information onto one line, formatting matters so indentation steps in. With json.dump(output_data, save_file, indent=4), clarity takes shape *(McKinney, 2017)*. That four space gap between levels turns dense text into something eyes can follow easily.

A closer look at classified_results.json reveals a correctly formatted JSON structure. From disorder to order the messy input has become an ordered hierarchy. Within this setup, emails sit separated in dedicated sections, whereas business related entries appear grouped side by side. Structure and compatibility are clearly present, meeting the criterion of saving information in a file through structured interchange methods.

Group 3: Data Structures Processing

Sorting Algorithm (Dictionaries):

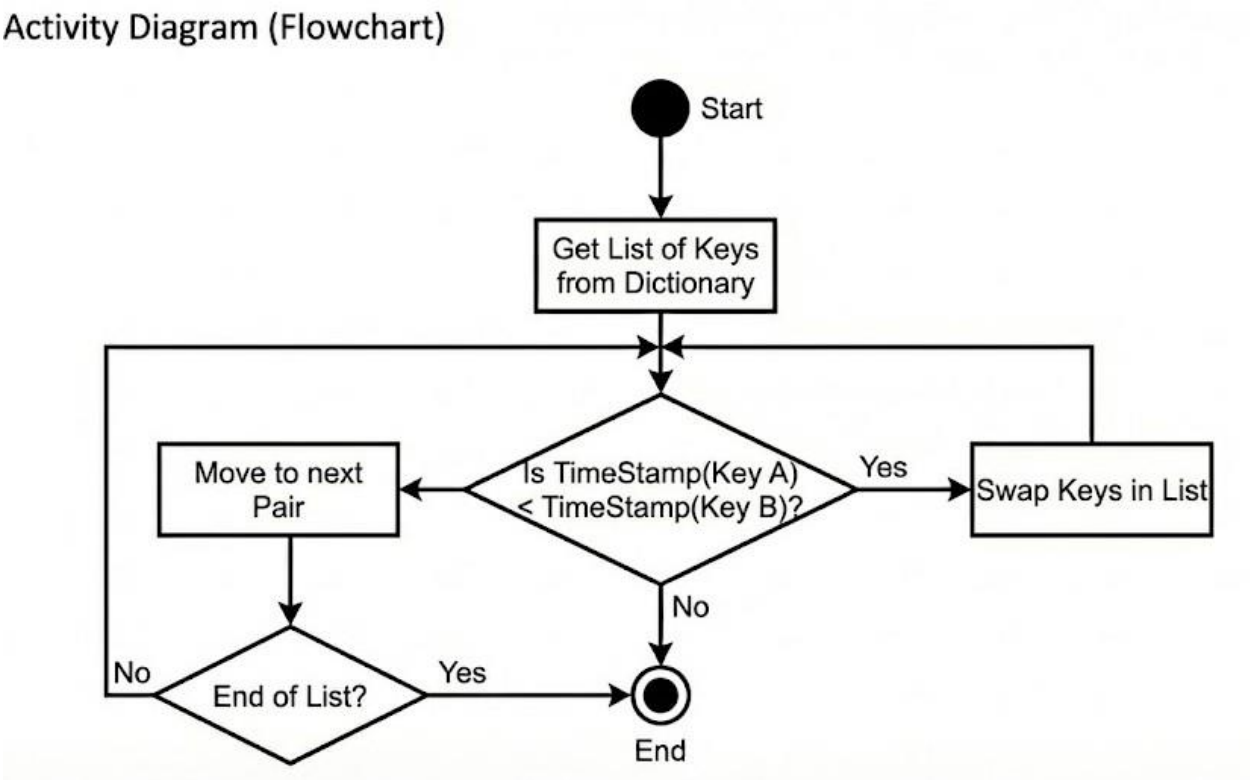


Figure 10: UML Activity Diagram for Sorting Algorithm (Dictionaries/TimeStamps).

A different kind of sorter handles Dictionaries - structures that pair Keys with Values. These collections do not follow order naturally, unlike numbered lists. Arranging them requires special steps, especially when sorting by something like Timestamp. First, the method pulls out all keys into a separate sequence because reordering directly inside the dictionary isn't possible. At the heart of the process sits a check: Is TimeStamp(Key A) < TimeStamp(Key B)? Each cycle examines when each key was recorded. When timing appears backward, movement shifts along the "Yes" route to swap key in list. Iteration persists until every key falls into time-based sequence. Such ordering proves necessary when handling records or event timelines within large-scale datasets.

Sorting Algorithm Logic (Arrays):

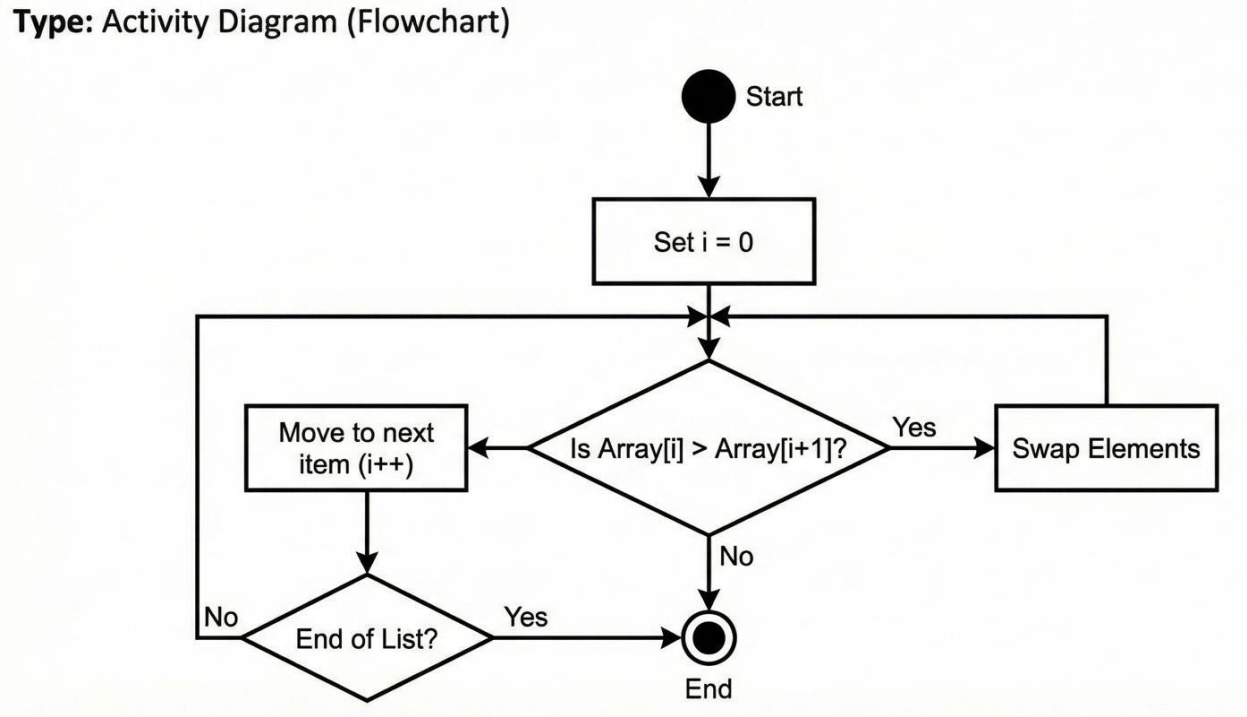


Figure 11: UML Activity Diagram for Sorting Algorithm (Arrays).

A closer look at the initial illustration among data structure visuals reveals a UML Activity Diagram outlining steps in a character sorting method. Focusing on arranging a sequence of words or names in alphabetical order meets one core evaluation criterion. Instead of abstract concepts, it maps out an everyday loop-driven approach common in programming basics. Commonly known within computing fields as Bubble Sort *(Hetland, 2017),* this technique works through repeated pairwise checks across elements. Because of its straightforward progression, educators frequently choose such models to demonstrate how arrays change during processing.

Step-by-Step Process Description:

Beginning at the Start Node a solid black circle the algorithm marks where processing kicks off. From there, a rectangle shows the first action: setting the index variable to zero (i = 0). That value directs attention to the initial position within the input array.

At the heart of the process sits the Decision Diamond, posing a key query: "Is Array[i] greater than Array[i+1]?". A check like this compares two adjacent characters alphabetically. What matters here is how one symbol stands relative to the next. Position plays a role each element faces its follower in sequence.

Ahead of its successor in alphabetical order like B coming before A the item triggers a shift. When that happens, elements exchange positions directly within storage. Movement occurs only if the first letter ranks later than the one after it. Step by step, greater values drift toward the endpoint. Direction changes when comparison favors the following character.

When items sit right where they need to be, swapping skips entirely - keeping everything just as it stands.

Iteration and Termination:

After comparing elements, movement continues toward advancing the index. Whether the list ends now becomes the deciding factor. When more items remain, execution returns to evaluate another set. Only when every element sits in order does the path lead forward. Reaching the final symbol marks completion not by assumption, but by design.

Searching Algorithm (Email Tree):

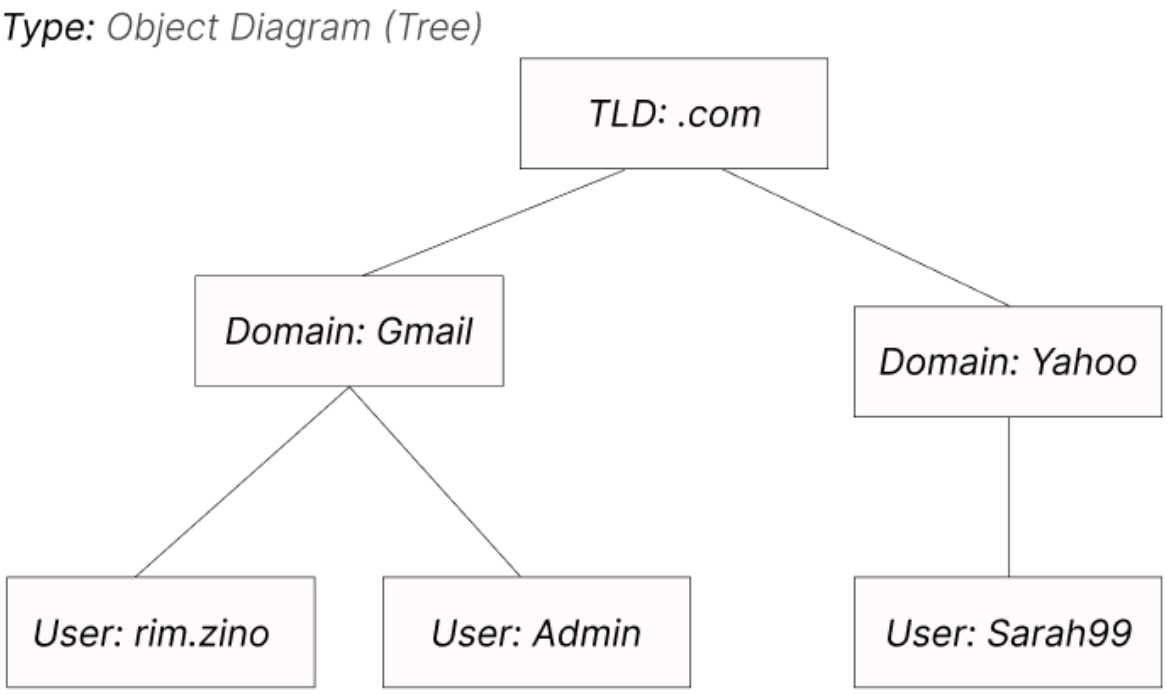


Figure 12: UML Object Diagram representing the Email Search Tree hierarchy.

A search method had to be built using a tree format *(Goodrich et al., 2013),* because of the third condition. Email addresses needed organizing into parts such as usernames, domain names, and top-level domains forming a layered system. This arrangement appears clearly in the UML diagram labeled 'group 3 uml 2.jpg'. From

broad categories at the top, information branches toward finer details below, much like limbs spreading on a tree.

Hierarchy Description:

Starting at the peak sits the TLD specifically .com a foundational label shaping how searches proceed. Its role defines the initial point for any system tracing a web address that concludes with .com. Progress unfolds downward only after touching this starting layer.

Below the root, information divides by provider type. One path leads to Gmail, another to Yahoo each forming a major offshoot. When processing queries, focusing on one domain skips the other entirely. Efficiency rises because only relevant sections require examination. Searching through less data saves time naturally.

At the bottom tier called leaf nodes are individual user accounts. “Rim.zino” appears under Gmail, alongside admin, both branching from that service. “Sarah99” connects instead to Yahoo’s segment. Each name marks a terminal point in the structure. Data access ends precisely at these entries.

A different way to look at it this setup speeds up finding prefixes by using branches. Starting from the top (.com), the process moves through nodes like Gmail before reaching “rim.zino”. Efficiency improves because instead of checking each entry one by one, the system follows a guided route. That shift changes how long searches take, cutting steps from O(n) down to roughly O(log n). Large collections of data benefit quietly, without fanfare.

Searching Algorithm (Location Tree Hierarchy):

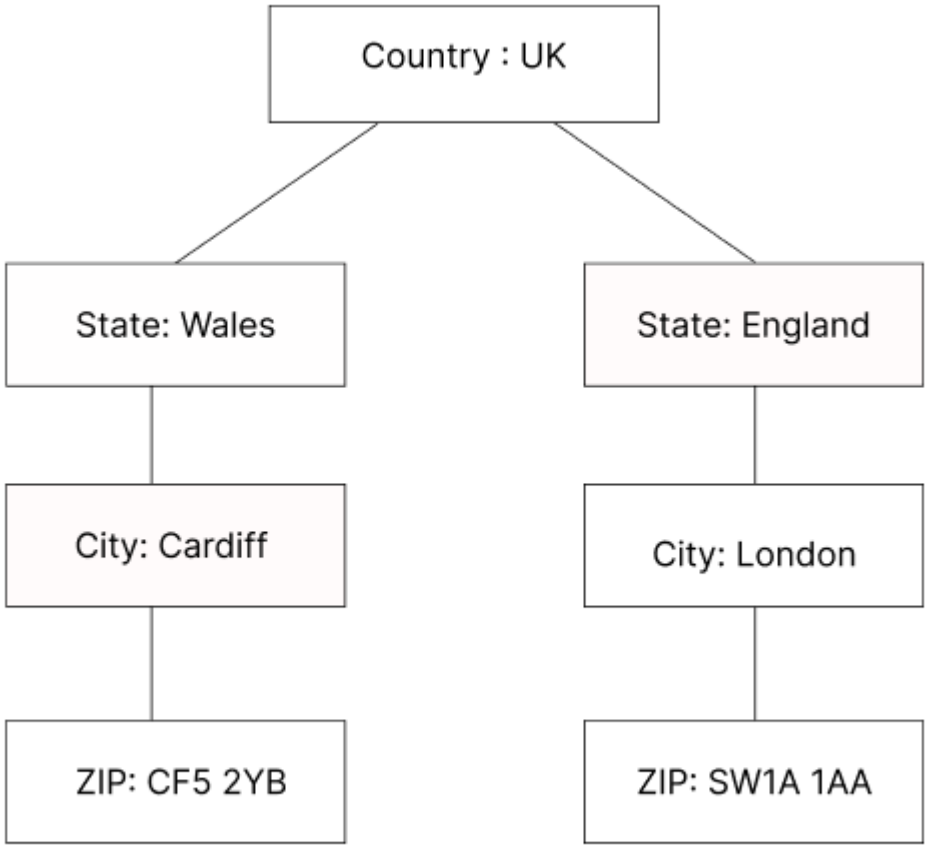


Figure 13: UML Object Diagram representing the Location Search Tree (UK Region).

Type: UML Object Diagram

A single path through regions shapes how location details get stored the setup follows country to state, then city down to postal zones. The layout takes form as a UML Object Diagram showing parent-child links across levels. Branching matters here; straight-line sequences slow things when handling worldwide entries. With split nodes at each tier, moving deeper into areas becomes quick, almost step-by-step, guided by structure rather than sequence.

A single framework splits into four layers, each nesting neatly within the next regions contain zones, which hold sectors, that in turn group smaller units. This setup links

locations through a straightforward hierarchy, where broader areas frame narrower ones below them

At the very top sits Country: UK this single node forms Level 1. Starting here allows the search process to begin. Everything else below links back to this nation's context. From this point forward, each piece extends within the same geographic boundary.

Starting from the top, the structure divides into two main areas - England and Wales. When someone looks up a place in London, the system ignores everything under Wales right away. One step down, the information narrows to specific urban centers. Cardiff appears under one path, London under another. Each choice cuts out irrelevant sections quickly.

At the very base of the structure sit individual entries Zip Codes like CF5 2YB or SW1A 1AA the smallest units in the system. These terminal points, known in computing as leaves, form level four of the hierarchy. Unlike upper layers, they lead nowhere beyond themselves. Branching stops here; no subdivisions follow. Each one acts as a final marker, holding detail without expansion.

A different route guides the search process here. Rather than checking each zip code one by one, the system moves step by step through layers: starting from country, then state, city, finally reaching zip. Because of this layered setup, fewer checks are needed at each stage. Performance stays strong, even as location counts grow into the millions.

Filtering Algorithm (Business Graph):

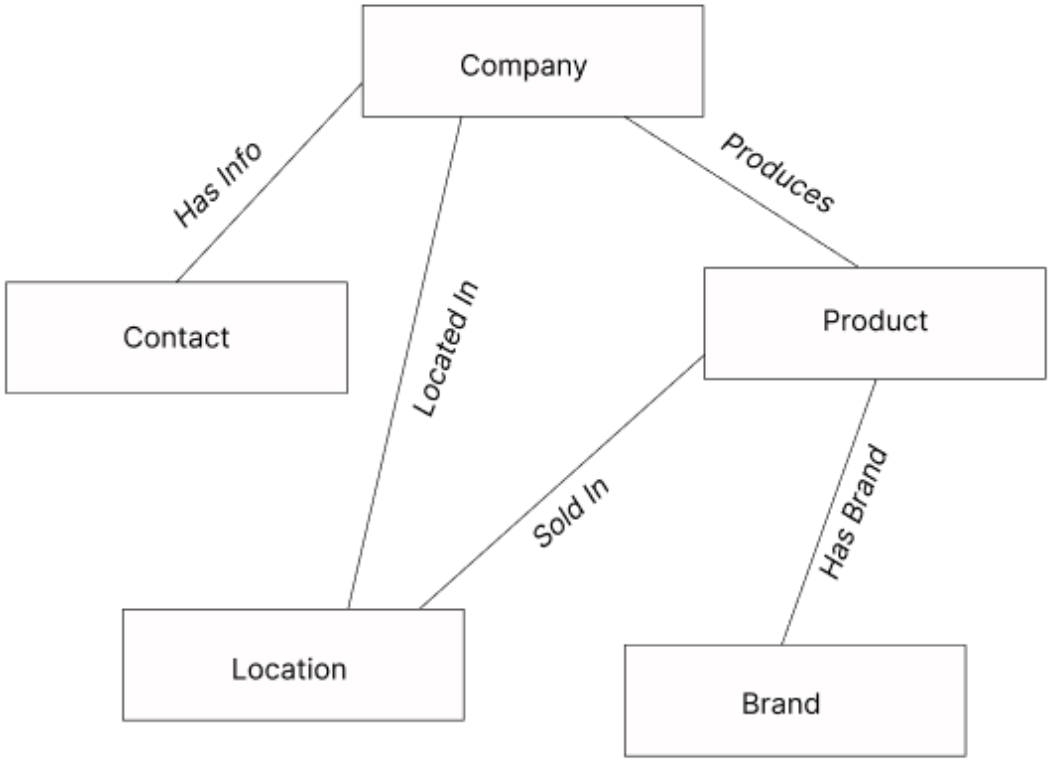


Figure 14: UML Class Diagram (Network Graph) for the business filtering algorithm.

Type: UML Class Diagram

What makes this last method different lies in how it handles linked information tying together items like products, costs, firms, labels, and places. Unlike tree models, which work well only when one item belongs under another, real-world connections often branch in several directions at once a store exists in a city but also sells many goods. For cases like these. These structures manage overlapping ties more naturally. Complexity does not disappear, yet representation becomes far closer to reality.

Picture a setup where things like Company, Product, Location, Brand, or Contact become dots on a map these are the Nodes *(Cormen et al., 2009)*. Between each dot, links appear not random ones, but meaningful ties called Edges that show how one piece of data connects to another.

A link appears directly between Company and Product, labeled "Produces," making connections visible. Because of this structure, a system can determine which items come from Company X. Questions about output find answers through the stated ties.

Imagine needing to find brands available in a certain city. Connections matter here specifically, how companies relate to places through "Located In" links, while products tie to regions via "Sold In." These relationships let the system check where things are offered. Take London: someone might look up which labels have items on sale there. The process moves from Location to Product, then onward to Brand, following those ties step by step. That flow makes targeted results possible.

Once a company appears via filters, contact data becomes available right away. This happens because the "Has Info" link ties each firm directly to its corresponding point of contact.

A web-like layout supports search methods that do not follow a straight path, such as scanning level by level. Because connections link points in multiple directions, processing skips around instead of moving step-by-step like in a sequence. Efficiency improves when handling intricate data demands common in enterprise analysis.

Conclusion

A closer look at how key software engineering methods work in Big Data contexts forms the core of this study. Through combining Python code with UML modeling, solutions took shape across varied challenges. One area focused on reaching into datasets effectively. Another tackled pulling specific information from large pools. A third handled intricate arrangements of data with precision. Each part built on practical execution rather than theoretical ideas. Tools were chosen based on fit, not popularity. Clarity guided decisions more than complexity ever could.

A solid foundation emerged in Group 1 through carefully defined access rules. Starting with registration and login systems, security took shape each step strengthened using Regex to guard passwords. Data stayed within reach thanks to search and display tools, yet remained under control. These steps together formed a practical approach to managing users safely.

A different angle emerged in Group 2 attention turned toward analyzing large datasets. Using a custom Python tool, raw text documents were processed effectively. Emails, carrying higher sensitivity, stood apart from business details like pricing or brand names, which posed fewer risks. These elements got sorted through automated labeling. One outcome was clear: turning messy inputs into organized JSON records made handling information more reliable. Structure mattered most when preparing data for long-term use.

To wrap up, Group 3 tackled the abstract challenges behind organizing information. Using UML illustrations ***(Larman, 2004)*** made it easier to see how Bubble Sort works step by step. Instead of just listing steps, they showed tree models that map email threads and physical sites. In the last part, a network-style diagram revealed patterns among items, firms, and places drawing from principles in graph math.

Working together, these methods form a full set of tools for guiding how data moves through software systems showing clear ability in security, storage, and detailed processing. While built differently, each plays a role in making sure operations run without breakdowns under pressure.

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). *Introduction to Algorithms*. 3rd ed. Cambridge, Mass.: MIT Press.

Goodrich, M.T., Tamassia, R. and Goldwasser, M.H. (2013). *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons.

Hetland, M.L. (2017). *Beginning Python: From Novice to Professional*. 3rd ed. New York: Apress.

Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design*. 3rd ed. Upper Saddle River, NJ: Prentice Hall.

McKinney, W. (2017). *Python for Data Analysis*. 2nd ed. Sebastopol, CA: O'Reilly Media.

Van Rossum, G. and Drake, F.L. (2009). *Python 3 Reference Manual*. Scotts Valley, CA: CreateSpace.